

ARQUITECTURA DE SOFTWARE

PRÁCTICO

Diccionario de Términos

Término	Significado simple
Productor	Envía un mensaje.
Broker	Recibe, guarda y entrega mensajes.
Cola	Espera los mensajes hasta que alguien los procese.
Consumidor	Recibe y procesa mensajes.
ACK	"Terminé de procesarlo".
DLQ	Cola donde apartamos mensajes rechazados.

Sincronismo vs. Asincronismo

Llamada telefónica (Síncrono)

El cliente llama y espera en la línea hasta obtener una respuesta. Si hay un error, lo sabe al instante. Ejemplo: HTTP request/response tradicional.

Bandeja de pedidos (Asíncrono)

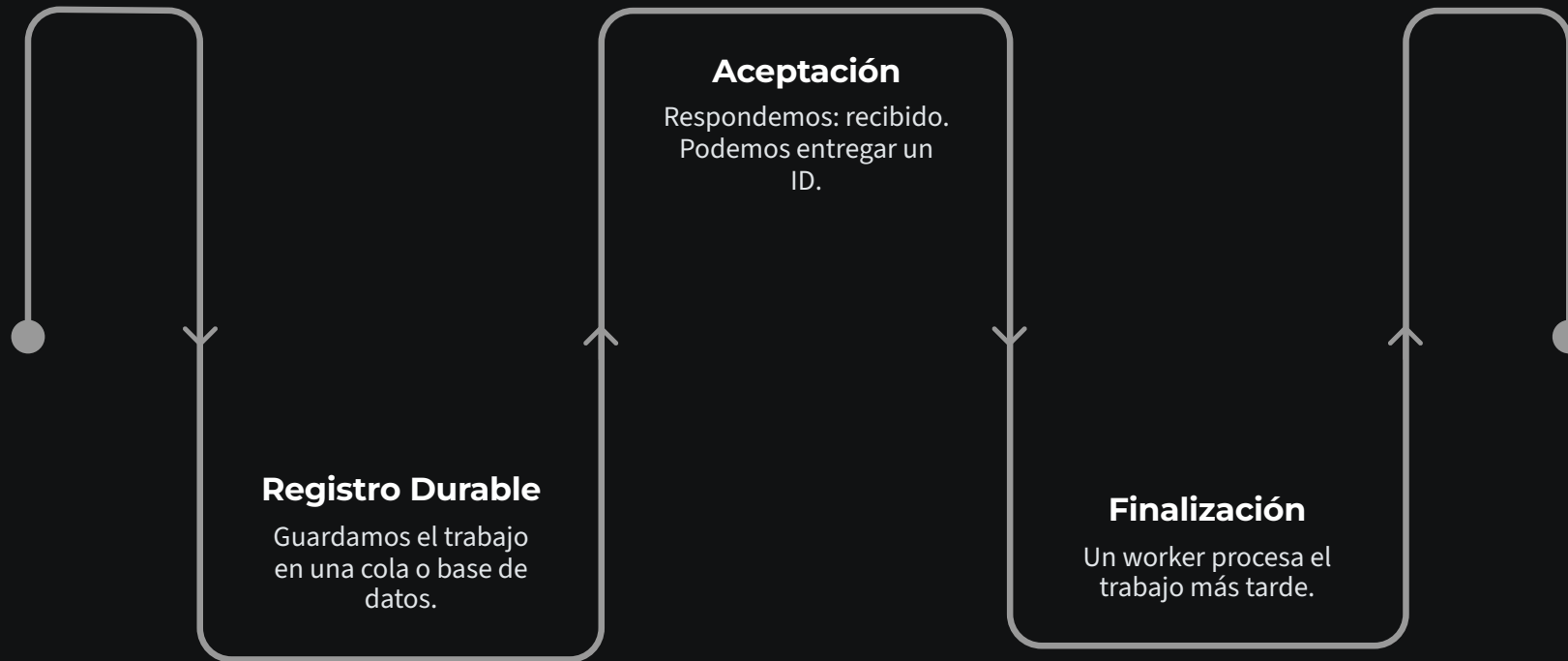
El cliente deja un pedido en una bandeja y sigue con otra tarea. Un worker lo toma cuando puede. El cliente no queda esperando.

En asincronismo, 'recibido' no significa 'terminado'. Significa: 'guardé el trabajo y lo procesaré después'.

La diferencia no es que algo sea más rápido. La diferencia es que el emisor no queda esperando.

Los Tres Momentos de un Trabajo Asíncrono

Un trabajo asíncrono es una tarea que no se termina en el momento en que la pides. Es como en un restaurante: haces tu pedido, te confirman que lo recibieron, lo preparan detrás de escena y te avisan cuando está listo.



Primero guardamos el trabajo. Después confirmamos que fue recibido. Por último, un worker lo procesa. Así no respondemos 'ok' por algo que todavía no quedó registrado.

En la Demo 1 vamos a ver el tercer momento: el worker toma el mensaje y lo confirma.

Los Tres Superdesacoples del Message Broker

Desacople Espacial

Productor y consumidor pueden estar en máquinas distintas.

Desacople Temporal

No tienen que estar encendidos al mismo tiempo.

Desacople de Ritmo

La cola absorbe un pico mientras el consumidor trabaja a su velocidad.

La actividad de QoS muestra el desacople de ritmo; no hace falta explicar más teoría ahora.

Docker

Ejecutar RabbitMQ aislado, sin ensuciar el SO:

```
docker run -d --name clase-rabbitmq \  
-p 5672:5672 -p 15672:15672 \  
-e RABBITMQ_DEFAULT_USER=user \  
-e RABBITMQ_DEFAULT_PASS=pass \  
rabbitmq:3-management
```

Panel de control

No vamos a instalar ni configurar RabbitMQ. Este comando levanta el broker que usa todo el material.

Panel: <http://localhost:15672> — Usuario: **user** — Contraseña: **pass**

Panel de RabbitMQ Manager — Queues

Hoy miramos solo Queues.

- Ready: mensajes esperando un consumidor.
- Unacked: mensajes entregados que todavía no terminaron.
- Total: Ready + Unacked.

El panel es nuestro observatorio: miramos qué hace el mensaje, no memorizamos pantallas.

productor.go — El Emisor

productor → cola-pedidos

Conecto con RabbitMQ

Creo o reutilizo cola-pedidos

1

2

3

4

Abro un canal

Envío 'pedido-101'

consumidor.go — ACK Manual

ACK no significa 'lo leí'. Significa 'terminé de procesarlo'.



VER DEMO 1: Haz Click Aquí

Ejercicios

1

Fair Dispatch — QoS = 1

Objetivo

Que el consumidor rápido reciba más trabajo cuando el lento sigue ocupado.

Consigna

Completá el TODO de `consumidor/main.go` para configurar QoS antes de `Consume`. Ejecutá dos consumidores y un productor. Probá primero sin tu cambio y luego con tu cambio.

2

Fanout Exchange — Publish/Subscribe

Objetivo

Que un pedido llegue a facturación y logística.

Consigna

Una tienda confirma un pedido. Facturación debe generar la factura y logística debe preparar el envío. Los programas ya crean el exchange y las colas. Completá primero el vínculo de facturación y publicá un pedido. Después completá el vínculo de logística y publicá otro: ahora ambos deben recibir una copia.

En QoS miramos reparto de trabajo; en Fanout miramos una copia del mismo evento para dos destinos.

Demo 2 — DLQ (Dead Letter Queue)

¿Qué pasa cuando un mensaje no se puede procesar?

Sin DLQ:

mensaje-1 → ACK

mensaje-2 → ERROR → ¿qué hacer?

mensaje-3 → Por el error anterior nunca se termina procesando.

Con DLQ:

mensaje-1 → ACK

mensaje-2 → NACK → pasa a DLQ (apartado), y puede seguir procesando ya que no bloquea

mensaje-3 → ACK

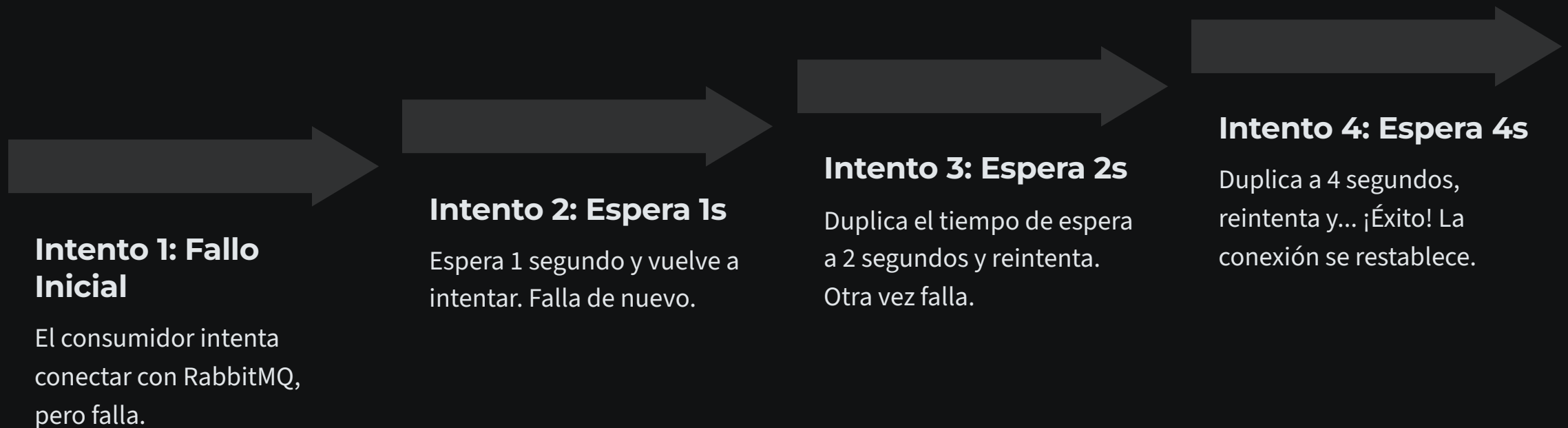
La DLQ es una cola de espera para mensajes problemáticos. No los arregla, solo los aparta para que la cola principal siga funcionando.

Después, un operador puede revisar la DLQ, entender qué salió mal y decidir qué hacer.

VER DEMO 2: Haz Click Aquí

Backoff Exponencial

Veamos cómo funcionan estos mecanismos de resiliencia para mantener nuestro sistema operativo, incluso cuando surgen problemas inesperados, como por ejemplo imagina que tu aplicación pierde la conexión con RabbitMQ. Así es como se recupera:



Esperar cada vez más protege al broker: evita golpearlo miles de veces mientras se está recuperando.

VER DEMO 3: Haz Click Aquí